

Program 1

Write a code in PL/SQL to develop a trigger that enforces referential integrity by preventing the deletion of a parent record if child records exist.

```

CREATE OR REPLACE TRIGGER
trg-prevent-department-delete
BEFORE DELETE ON departments
FOR EACH ROW
DECLARE
    v-child-count NUMBER;
BEGIN
    SELECT COUNT (*)
    INTO v-child-count
    FROM employees
    where department_id = :OLD.department_id;
    IF v-child-count > 0 THEN
        RAISE -APPLICATION_ERROR (-20001, 'cannot delete
        department because employees exist in the department,');
    END IF;
END;

```

Program 2

Write a code in PL/SQL to create a trigger that checks for duplicate values in a specific column and raises an exception if found.

CREATE OR REPLACE TRIGGER.

Trg-check-duplicate-employee-mail

BEFORE INSERT OR UPDATE ON Employees.

FOR EACH ROW

DECLARE

V-COUNT NUMBER;

BEGIN

SELECT COUNT(*) INTO V-COUNT FROM employees.

WHERE mail = :NEW.mail AND employee-id !=

:NEW.employee-id)

IF V-COUNT > 0 THEN

RAISE_APPLICATION_ERROR(-20002, 'Duplicate mail
detected');

END IF;

END;

Program 3

Write a code in PL/SQL to create a trigger that restricts the insertion of new rows if the total of a column's values exceeds a certain threshold.

```
CREATE OR REPLACE TRIGGER tng-limit-total-salary
BEFORE INSERT OR UPDATE ON employees
FOR EACH ROW
```

```
DECLARE
```

```
  v-total NUMBER;
```

```
  v-threshold NUMBER := 1000000;
```

```
BEGIN
```

```
  SELECT NVL(SUM(salary), 0) INTO v-total FROM employees;
```

```
  IF INSERTING THEN
```

```
    total := v-total + :NEW.salary;
```

```
  ELSIF UPDATING THEN
```

```
    v-total := v-total - :OLD.salary + :NEW.salary;
```

```
  ENDIF;
```

```
  IF v-total > v-threshold THEN
```

```
    RAISE_APPLICATION_ERROR (-20003, 'Total salary exceeds  
the allowed limit.');
```

```
  END IF;
```

```
END;
```

Program 4

Write a code in PL/SQL to design a trigger that captures changes made to specific columns and logs them in an audit table.

```
CREATE TABLE employees_audit (  
  audit_id NUMBER GENERATED ALWAYS AS IDENTITY,  
  employee_id NUMBER,  
  old_salary NUMBER,  
  new_salary NUMBER,  
  old_job_id VARCHAR(20),  
  new_job_id VARCHAR(20),  
  changed_by VARCHAR(50),  
  changed_on DATE  
);
```

```
CREATE OR REPLACE TRIGGER trg_audit_employee_changes  
AFTER UPDATE OF salary, job_id ON employees  
FOR EACH ROW
```

```
BEGIN  
  INSERT INTO employees_audit (  
    employee_id,  
    old_salary,  
    new_salary,  
    old_job_id,  
    new_job_id,  
    changed_by,  
    changed_on  
  )  
  VALUES (  
    :old_employee_id,  
    :old_salary,  
    :new_salary,  
    :old_job_id,  
    :new_job_id,  
    USER,  
    SYS DATE  
  );  
END;
```

END ;

Program 5

Write a code in PL/SQL to implement a trigger that records user activity (inserts, updates, deletes) in an audit log for a given set of tables.

```
CREATE TABLE audit_log (  
  audit_id NUMBER GENERATED ALWAYS AS IDENTITY,  
  table_name VARCHAR2(60),  
  operation VARCHAR2(10),  
  record_id NUMBER,  
  changed_by VARCHAR2(50),  
  changed_on DATE  
);  
  
DECLARE  
  v_operation VARCHAR2(10);  
  v_record_id NUMBER;  
  
BEGIN  
  IF INSERTING THEN  
    v_operation := 'INSERT';  
    v_record_id := :NEW.employee_id;  
  
  ELSIF UPDATING THEN  
    v_operation := 'UPDATE';  
    v_record_id := :OLD.employee_id;  
  
  ELSEIF DELETING THEN  
    v_operation := 'DELETE';  
    v_record_id := :OLD.employee_id;  
  
  END IF;  
  
  INSERT INTO audit_log  
    (table_name,  
     operation,  
     record_id,  
     changed_by,  
     changed_on)  
  VALUES (  
    'EMPLOYEES',  
    v_operation,  
    v_record_id,  
    USER,  
    SYSDATE  
  );  
END;
```

Program 7

Write a code in PL/SQL to implement a trigger that automatically calculates and updates a running total column for a table whenever new rows are inserted.

```
CREATE TABLE Sales
```

```
Sale_id NUMBER GENERATED ALWAYS AS IDENTITY
```

```
Sale_amount NUMBER,
```

```
running_total NUMBER.
```

```
);
```

```
CREATE OR REPLACE TRIGGER trg_update_running_total  
AFTER INSERT ON Sales
```

```
FOR EACH ROW
```

```
DECLARE
```

```
v_total NUMBER;
```

```
BEGIN
```

```
SELECT NVL(SUM(Sale_amount), 0)
```

```
INTO v_total
```

```
FROM Sales;
```

```
UPDATE Sales
```

```
SET running_total = v_total
```

```
WHERE Sale_id = NEW.Sale_id;
```

```
END;
```

Program 8

Write a code in PL/SQL to create a trigger that validates the availability of items before allowing an order to be placed, considering stock levels and pending orders.

```
CREATE TABLE items (
  item_id NUMBER PRIMARY KEY,
  item_name VARCHAR2(100),
  stock_quantity NUMBER
);
```

```
CREATE TABLE orders (
  order_id NUMBER GENERATED ALWAYS AS IDENTITY
  PRIMARY KEY,
  item_id NUMBER,
  order_quantity NUMBER,
  order_date DATE
);
```

```
DECLARE
```

```
  v_stock NUMBER;
```

```
  v_pending NUMBER;
```

```
BEGIN
```

```
  SELECT NVL (stock_quantity, 0)
```

```
  INTO v_stock
```

```
  FROM items
```

```
  WHERE item_id = :NEW.item_id;
```

```
  SELECT NVL (SUM (order_quantity), 0)
```

```
  INTO v_pending
```

```
  FROM orders
```

```
  WHERE item_id = :NEW.item_id;
```

```
  IF v_stock + v_pending < :NEW.order_quantity THEN
```

```
    RAISE_APPLICATION_ERROR (-20004, 'Insufficient stock available to fulfill the order.');
```

```
  END IF;
```

```
END;
```

Evaluation Procedure	Marks awarded
PL/SQL Procedure(5)	5
Program/Execution (5)	5
Viva(5)	5
Total (15)	15
Faculty Signature	